Practice Exam - Programming for Minor Computational Science

Name:

.....

Log in:

You will make this exam using the online editor. To access it:

- Go to **mcs.proglab.nl**.
- Enter the exam code: **abc123**
- Enter your **student number**.
- Keep your student card or ID on your desk.

Goal:

- This digital exam consists of 5 assignments where you will write short Python programs.
- You need at least 3 assignments correct in order to pass the exam.
- You will be **evaluated solely on the correctness** of your solutions. Code design, comments, and style are not important, so you do not need to worry about them.
- The editor has a Checkpy button that allows you to verify your solutions during the exam for your personal use. Note that Checkpy results will not influence your actual grade, as all grading is done manually. So, even if your solutions do not pass Checkpy, they may still be accepted during the grading process.

Constraints:

- You may use the course website as a resource (open mcs.proglab.nl again in a new tab after logging in).
- You may use a printed version of the cheatsheet during the exam.
- Do not use external modules such as `numpy`, `csv`, or others unless the assignment specifically says you can.

Submit:

When you have completed all assignments:

- use the **submit button** on the website to ensure your final edits are committed,
- **please fill out the evaluation** (link in submit window), this helps us a lot to improve the course,
- close the browser,
- hand in this exam sheet **with your name on it**.

Assignment 1: Dud

Write a function `dud(n)` that returns a list of "dud" numbers between 1 and n (excluding n). Dud numbers are numbers that satisfy the following condition: the number is divisible by 3, and the number plus 2 is divisible by 5. For example, 3 is a valid dud number because 3 is divisible by 3 and 3 + 2 = 5 is divisible by 5.

Example

```
print(dud(200))
```

Expected Output

```
[3, 18, 33, 48, 63, 78, 93, 108, 123, 138, 153, 168, 183, 198]
```

Assignment 2: Replace order

You have a dictionary from a webshop containing customer orders. The keys are the names of the customer, the values are the products they ordered. Some products are out of stock and you want to automatically replace them with an alternative.

Write a function `replace_product(orders, replacements)` that updates the dictionary of `orders`, based on a given dictionary of `replacements`. The function should return the updated version of the orders dictionary. If a product is not found in the replacements dictionary, it should remain unchanged.

Example Usage

```
orders = {
    "Aisha Khan": ["Kindle", "Trailmix"],
    "Lucas van Dijk": ["Fitbit", "Gödel, Escher, Bach"],
    "Mei Lin": ["Unplugged"],
    "Omar Haddad": ["McLaren 750s", "Fitbit"],
    "Sofia Rossi": ["Trailmix", "Kindle", "Unplugged"],
    "Diego Martínez": ["Gödel, Escher, Bach"],
    "Anna Kowalska": ["Fitbit", "Trailmix"],
    "Kwame Mensah": ["Kindle", "McLaren 750s"],
    "Yuki Tanaka": ["Unplugged", "Fitbit"],
    "Fatima El-Sayed": ["Trailmix", "Gödel, Escher, Bach"]
}

replacements = {
    "Unplugged": "Nevermind",
    "Trailmix": "Popcorn",
    "McLaren 750s": "Toyota Yaris"
}

updated_orders = replace_product(orders, replacements)
print(updated_orders)
```

Expected Output

```
{
    'Aisha Khan': ['Kindle', 'Popcorn'],
    'Lucas van Dijk': ['Fitbit', 'Gödel, Escher, Bach'],
    'Mei Lin': ['Nevermind'],
    'Omar Haddad': ['Toyota Yaris', 'Fitbit'],
    'Sofia Rossi': ['Popcorn', 'Kindle', 'Nevermind'],
    'Diego Martínez': ['Gödel, Escher, Bach'],
    'Anna Kowalska': ['Fitbit', 'Popcorn'],
    'Kwame Mensah': ['Kindle', 'Toyota Yaris'],
    'Yuki Tanaka': ['Nevermind', 'Fitbit'],
    'Fatima El-Sayed': ['Popcorn', 'Gödel, Escher, Bach']
}
```

Assignment 3: Filter

Write a function `select_adults(people, ages)` that receives two lists of equal length: one with names (people) and one with corresponding ages (ages). The function should return a new list containing only the names of people who are 18 years or older.

Example

```
people = ["Jan", "Piet", "Anne", "Marieke", "Robin", "Jeroen",
    "Femke", "Kim", "Bas", "Noor", "Lieke", "Sam"]
ages = [12, 52, 15, 29, 18, 41, 26, 4, 47, 16, 24, 13]
print(select_adults(people, ages))
```

Expected Output

```
['Piet', 'Marieke', 'Robin', 'Jeroen', 'Femke', 'Bas', 'Lieke']
```

Assignment 4: Anagrams

Two words are anagrams of each other if they consist of exactly the same letters, like `listen` and `silent`.
Write a function `anagram_pairs(words)` that returns a list of all pairs of words in `words` that are anagrams of each other. Each pair is a tuple and appears only once in the list. The order of the pairs, and of the two words within a pair, does not matter.

Example Usage

```
words = ["listen", "stone", "google", "silent", "notes", "banana",
         "enlist", "tones", "apple", "onset", "inlets"]
print(anagram_pairs(words))
```

Expected Output

```
[('listen', 'silent'), ('stone', 'tones'), ('notes', 'stone'),
 ('notes', 'tones'), ('notes', 'onset'), ('enlist', 'listen'),
 ('enlist', 'silent'), ('enlist', 'inlets'), ('onset', 'stone'),
 ('onset', 'tones'), ('inlets', 'listen'), ('inlets', 'silent')]
```

Assignment 5: Dice

Write a function `dice_sum(n, k)` that calculates in how many ways each possible sum can occur when rolling `k` dice with `n` sides each (numbered 1 to `n`). The function should not simulate dice rolls, but compute all **possible combinations** of the dice.

It should return a dictionary where the keys are the possible sums, and the values are the number of combinations that produce that sum.

For example, with two six-sided dice there is only one way to get a sum of 2 (1+1), but six ways to get a sum of 7.

Example Usage

```
print(dice_sum(6, 2))  # two six-sided dice
print(dice_sum(4, 3))  # three four-sided dice
```

Expected Output

```
{2: 1, 3: 2, 4: 3, 5: 4, 6: 5, 7: 6, 8: 5, 9: 4, 10: 3, 11: 2, 12: 1}
{3: 1, 4: 3, 5: 6, 6: 10, 7: 12, 8: 12, 9: 10, 10: 6, 11: 3, 12: 1}
```